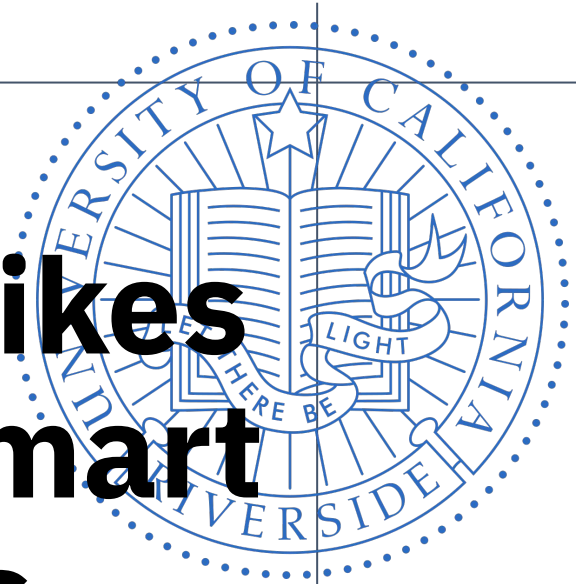


# Tracking Network Traffic Spikes and Anomaly Detection in Smart Security Camera Systems.



Jeremie Rubio, Karam Kambo

[jrubi065@ucr.edu](mailto:jrubi065@ucr.edu), [kkamb001@ucr.edu](mailto:kkamb001@ucr.edu)

August 29th, 2026



# Python libraries and Tools

## Data Manipulation:

- Pandas
- numpy
- glob
- os

## Signal Processing:

- SciPy (fft, kurtosis)
- matplotlib
- seaborn

## Machine Learning:

- SKlearn (labelencoder, standardscalar, KMeans, metric functions)



# Motivation & The Real World Problem

## **The Threat:**

Smart security cameras generate continuous, high-volume data streams, making them prime targets for malware like Mirai and BASHLITE.

## **The Goal:**

Accurately differentiate between harmless high traffic events and failures like DDoS attacks or device hijackings.

## **The Challenge:**

Beating an accuracy paradox. A model that guesses normal traffic every time is useless, even if it scores high accuracy.



# Data Choice & Intuition

**Source:** N-BaIoT Dataset via UCI Machine Learning Repository

<https://archive.ics.uci.edu/dataset/442/detection+of+iot+botnet+attacks+n+baiot>

**Scope:**

100,000 tabular time-series samples pulled specifically from Provision PTZ Security Camera Data.

**Inputs:**

115 Continuous statistical features (Average packet size, count, variance)

**Target Label:**

0 (Normal baseline traffic) vs 1 (Anomalous Mirai Attack)

|                    | average   | min  | max        | std       |
|--------------------|-----------|------|------------|-----------|
| MI_dir_L5_weight   | 7.765     | 1.0  | 67.260     | 12.501    |
| MI_dir_L5_mean     | 98.785    | 60.0 | 895.336    | 42.166    |
| MI_dir_L5_variance | 12220.904 | 0.0  | 491805.787 | 22941.558 |
| MI_dir_L3_weight   | 8.841     | 1.0  | 86.151     | 14.916    |
| MI_dir_L3_mean     | 98.733    | 60.0 | 711.845    | 40.690    |
| MI_dir_L3_variance | 12249.129 | 0.0  | 464500.556 | 22357.547 |
| MI_dir_L1_weight   | 11.215    | 1.0  | 115.058    | 18.616    |
| MI_dir_L1_mean     | 100.074   | 60.0 | 590.000    | 38.631    |
| MI_dir_L1_variance | 13258.945 | 0.0  | 179966.158 | 21310.354 |
| MI_dir_L0.1_weight | 27.833    | 1.0  | 149.385    | 32.899    |

# Data Preprocessing

## Data Cleaning:

Verified absolutely zero missing or infinite values

## Feature Scaling:

Applied StandardScalar across all 115 numeric columns

## Mathematical Necessity:

Forced the feature means to 0.0 and standard deviations to 1.0 to prevent large values from overpowering distance based algorithms

```
import numpy as np

nulls = df.isnull().sum()
print("Total missing values:", nulls.sum())
print(nulls[nulls > 0])
print("Infinite values:", np.isinf(df.select_dtypes("number")).sum().sum())
```

```
... Total missing values: 0
Series([], dtype: int64)
Infinite values: 0
```

```
df = df.replace([np.inf, -np.inf], np.nan)
df = df.fillna(df.median(numeric_only=True))

print("Missing after cleaning:", df.isnull().sum().sum())
print("Shape:", df.shape)
```

```
... Missing after cleaning: 0
Shape: (98514, 116)
```

```
Mean after scaling: 0.0
Std after scaling: 1.0
```

|   | MI_dir_L5_weight | MI_dir_L5_mean | MI_dir_L5_variance | MI_dir_L3_weight | MI_dir_L3_mean | MI_dir_L3_variance |
|---|------------------|----------------|--------------------|------------------|----------------|--------------------|
| 0 | -0.541136        | -0.919809      | -0.532700          | -0.525645        | -0.951926      | -0.532700          |
| 1 | -0.541136        | 11.649605      | -0.532700          | -0.525645        | 12.073544      | -0.532700          |
| 2 | -0.464582        | 11.649605      | -0.532700          | -0.460349        | 12.073544      | -0.532700          |
| 3 | -0.541136        | -0.919809      | -0.532700          | -0.525645        | -0.951926      | -0.532700          |
| 4 | -0.462052        | -0.776698      | -0.531131          | -0.459062        | -0.803961      | -0.532700          |

5 rows x 116 columns

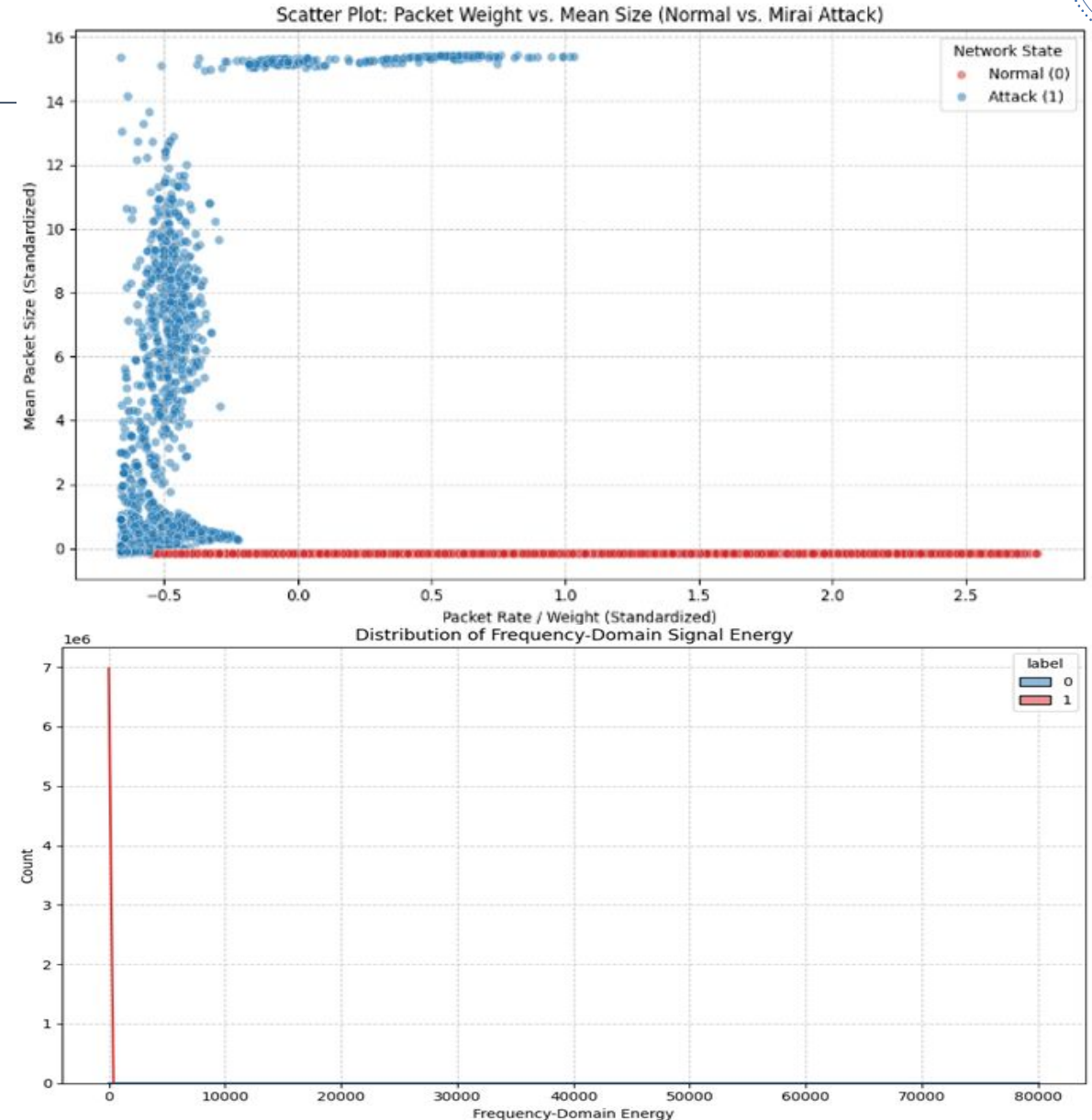
# Visualizing Threats

## Scatter Plot:

Mirai Traffic forms a dense, distinct cluster of extreme packet frequencies with identical tiny packet sizes.

## Histogram (FFT):

Attack data stacks into vertical spikes at zero frequency domain energy due to identical spammed packets.



# Machine Learning (Clustering)

## Algorithm:

Unsupervised K-Means++ Clustering

## Tuning:

Stripped target labels for blind training:

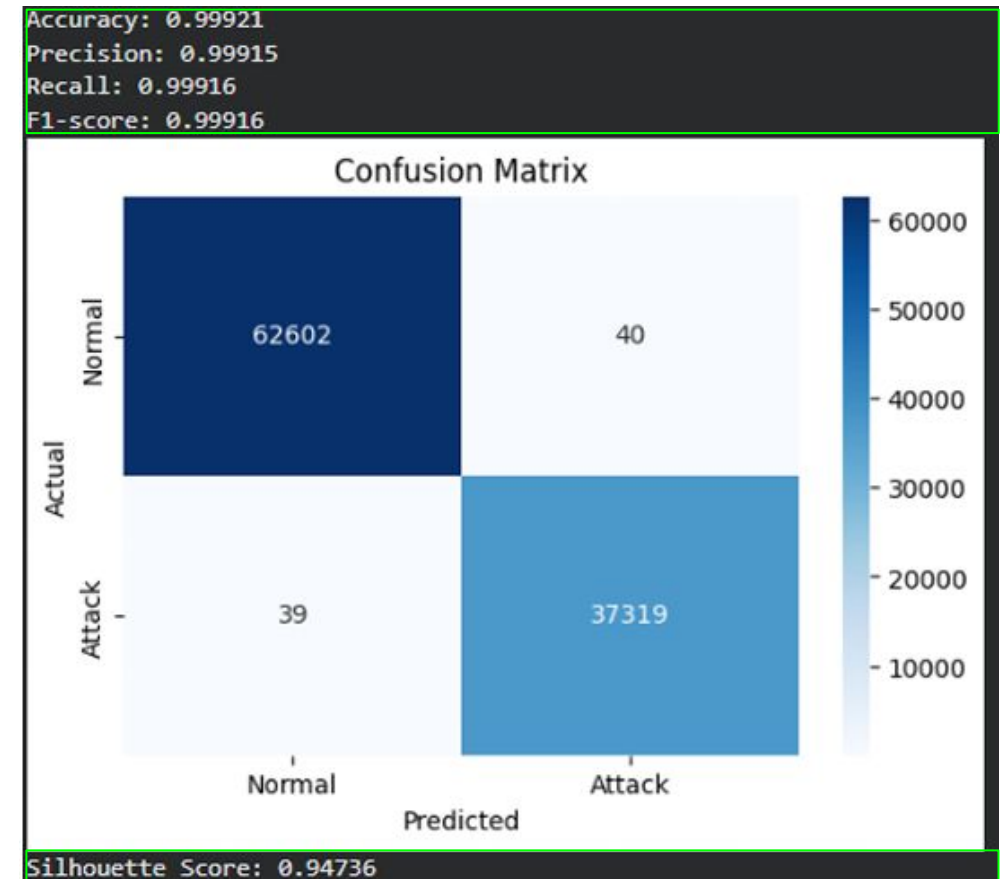
- Utilized K-Means++ initialization to prevent extreme outliers from trapping the centroids

## Baseline metrics:

99.9% across Accuracy, Precision, Recall, and F1-Score

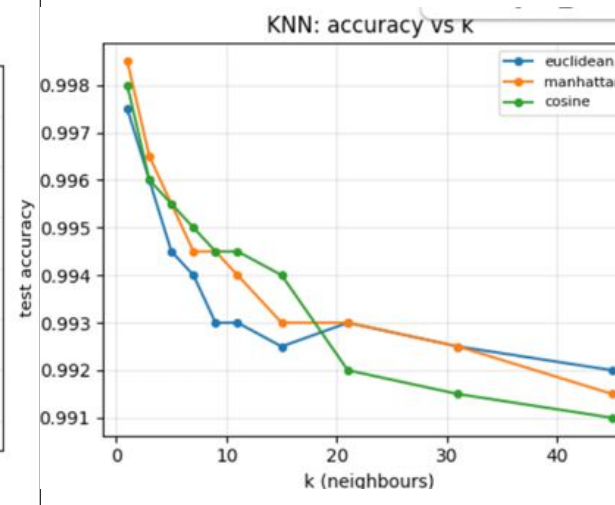
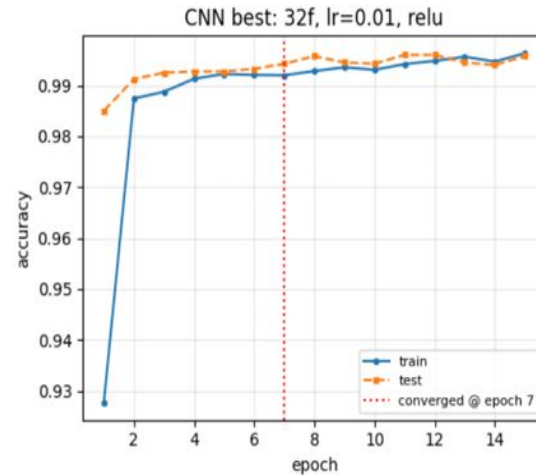
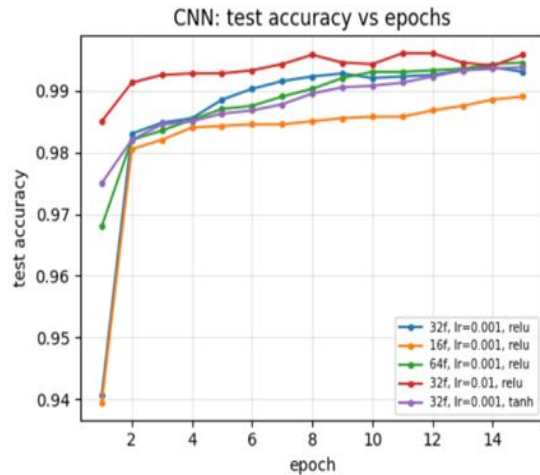
## Separation:

Achieved a highly dense Silhouette Score of .947





# Comparative Neural Networks & Classification



## LSTM Time Series:

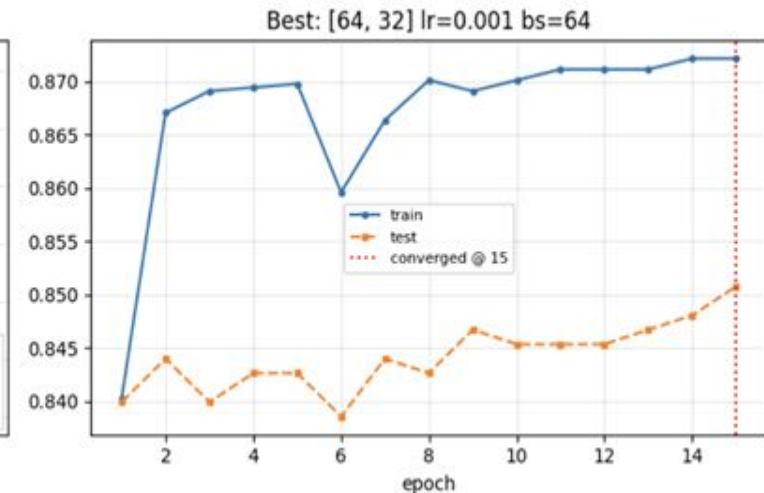
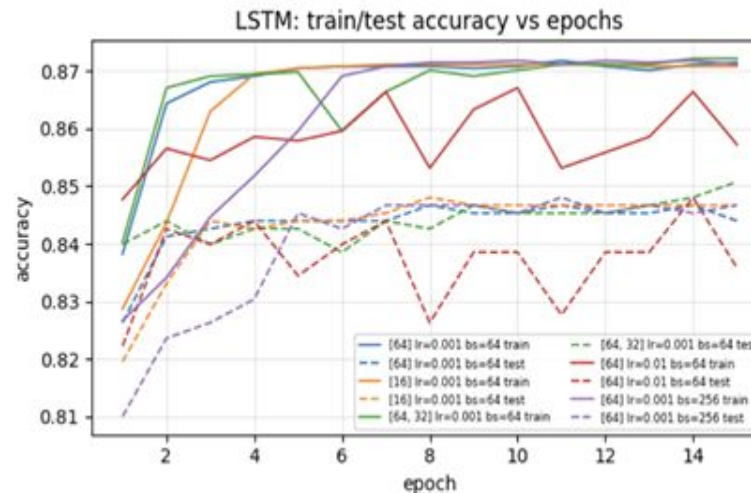
64 and 32 layer configuration with  $lr=0.001$  and a batch size of 64 converged at epoch 15 (0.8507 accuracy)

## CNN Classification:

Converged at epoch 7 using 32 filters,  $lr=0.01$ , and ReLU activation, achieving 0.9968 test accuracy.

## KNN Classification:

Tested Euclidean, Manhattan, and Cosine metrics; peaked at  $k=1$  with Euclidean distance (0.9975 accuracy).





# Hypothesis Testing & Feature Reduction

## Methodology:

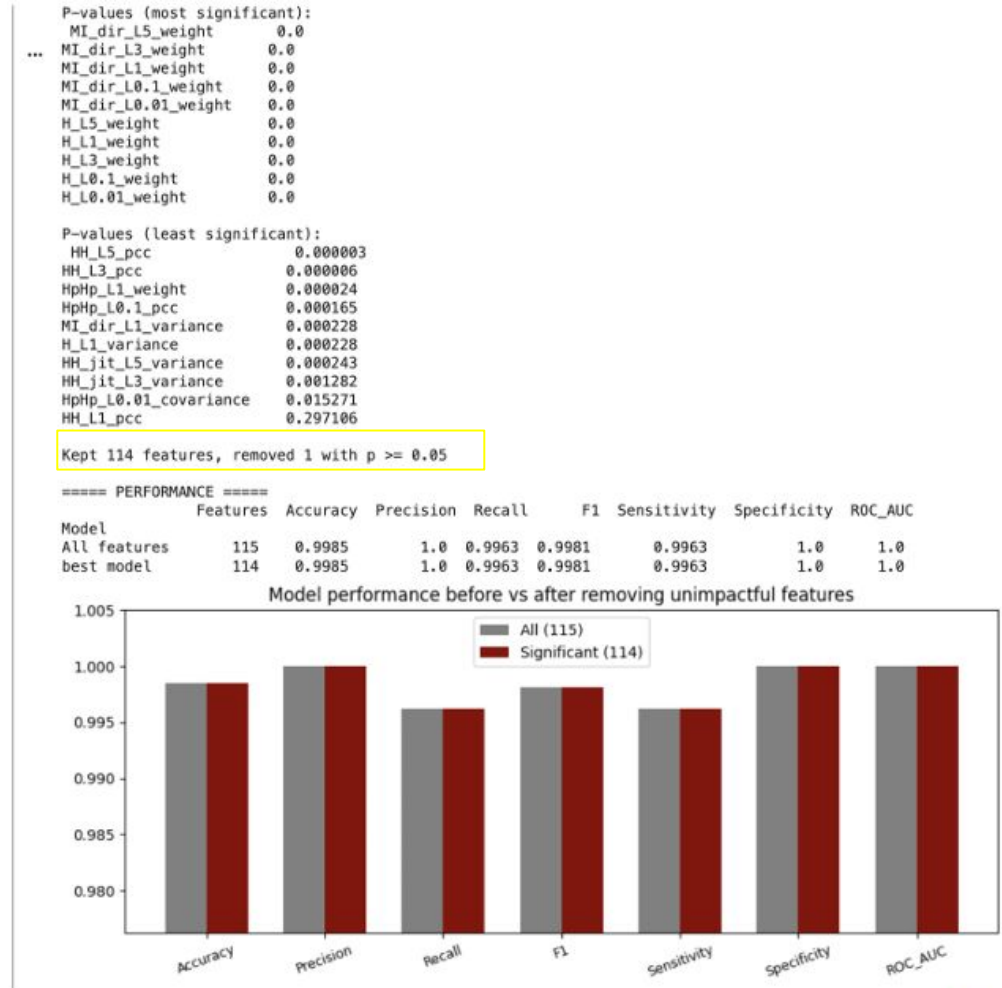
Calculated exact P-Values across all 115 original features

## Execution:

Removed the least significant feature (HH\_L1\_pcc,  $p=.0297106$ ) while retaining the remaining 114.

## Impact:

Performance remained mathematically identical (Accuracy: 0.9985, Precision: 1.0, Recall: 0.9963)



# Dimensionality Reduction (PCA)

## Configuration:

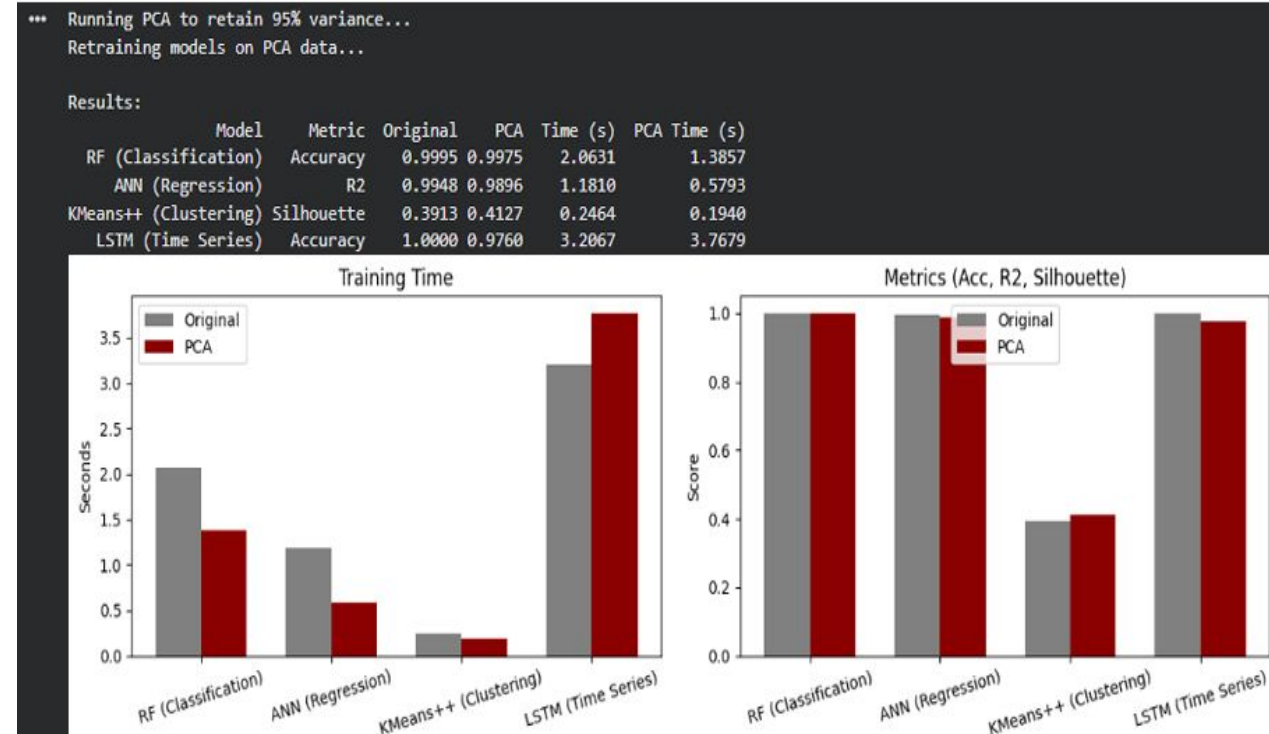
Applied Principal Component Analysis (PCA) to retain exactly 95% variance.

## Speed Optimization:

Cut ANN training time in half (1.18s down to 0.57s) and Random Forest time from 2.06s to 1.38s

## Metric Stability:

Predictive performance barely moved (RF accuracy shifted trivially from 0.9995 to 0.9975)





# Conclusions & Limitations

**Success:** Distance-based clustering and Neural Networks successfully learned the physical mechanics of IoT threats, beating the accuracy paradox.

**The PCA Trade-Off (Limitation):** While PCA optimized computing speeds, it caused a complete loss of interpretability.

**Next Steps:** Future implementations must balance the speed of PCA against the need to explain exactly which physical network trait (e.g., packet weight) triggered the security alarm.